

LLM Notes

2026 Spring

Day 1: 损失函数与反向传播

deeplearning 基础

主讲: Aaron

Fudan University

今日摘要

这一讲主要复习损失函数与反向传播, 做一些简单的公式推导

1.1 损失函数

损失函数是神经网络甚至所有监督学习算法的根本所在, 所有监督学习任务均需要模型预测标签 $\hat{y}$ 与真实标签 y 。

而损失函数 $L(y, \hat{y})$ 用一个标量衡量两者之间的 gap, 作为一切训练的依据。

定义

定义 1.1.1 (MSE Loss). 针对回归任务, 一般使用均方误差 (MSE) 损失。

设有 N 个样本, 输出维度为 D

$$\mathcal{L} = \sum_{d=1}^D (y_d - \hat{y}_d)^2$$

$\hat{y}_d$ 是模型对该样本第 d 维的预测值, y_d 是对应真实连续标签。

分析

内层求和用于累加单个样本所有输出维度的误差平方; 平方操作会放大预测与真值之间的差值, 对偏离较大的样本施加更强惩罚; 同时平方保证了误差间不会抵消, 并且利于求导计算。

定义

定义 1.1.2 (Cross Entropy Loss). 针对多分类任务，一般使用交叉熵损失，设有 K 个类别

$$\mathcal{L} = \sum_{k=1}^K y_k \log(\hat{y}_k)$$

$\hat{y}_k$ 是预测标签概率， y_k 是真实概率 (0 or 1)

分析

内层的求和实际上会退化为单值，因为 y_{ik} 在第二个维度上是 one-hot 编码（样本 i 只能属于单一类别）

1.1.1 softmax 的作用

softmax 函数本身只是一个归一化函数，用于将一组数值从实数空间映射到概率空间：

如有一个向量 $[a_1, a_2, \dots, a_K]$

$$\text{softmax}(a_i) = \frac{\exp(a_i)}{\sum_{k=1}^K \exp(z_k)}$$

其在神经网络中的意义主要在于：一般模型输出的一组数值都不会特地归一化，而是直接的 logits 分数（实数范围内任意取值），为了让其具备实际意义，需要对模型输出的 logits 分数做归一化得到概率分布。

同时指数运算会拉大不同分量的数值差距，强化优势类别的区分度，适配分类任务的学习目标。

1.1.2 多分类交叉熵损失与 Softmax

在多分类任务中，模型最后一层（输出层）通常输出一个维度为类别数 K 的实数值向量，称为 **Logits**，记作 $z = [z_1, z_2, \dots, z_K]^T$ 。这些 Logits 是未归一化的“分数”。

为了将其转换为一个和为 1 的概率分布 $\hat{y}$ ，我们需要使用 **Softmax 函数**：

$$\hat{y}_k = \text{Softmax}(z_k) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K$$

此时，单个样本的交叉熵损失（结合真实标签的 one-hot 向量 $\mathbf{y}$ ）可写为：

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{y}_k) \quad (1.1.1)$$

$$= - \sum_{k=1}^K y_k \log \left(\frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)} \right) \quad (1.1.2)$$

$$= - \sum_{k=1}^K y_k \left(z_k - \log \sum_{j=1}^K \exp(z_j) \right) \quad (1.1.3)$$

$$= -z_c + \log \sum_{j=1}^K \exp(z_j), \quad \text{其中 } c \text{ 为真实类别} \quad (1.1.4)$$

分析

编程实现中的 **softmax** 在 PyTorch 的 **CrossEntropyLoss** 中，通常将 Logits 直接输入损失函数，因为其内部已高效集成了 Log-Softmax 计算。不过理论推导中，Softmax 不可忽略，它使得反向传播的初始误差项 $\delta^{(L)}$ 可以推导为简洁的 $\hat{\mathbf{y}} - \mathbf{y}$ 形式。

1.2 反向传播算法

前章节的 loss function 主要回答优化目标的问题，而反向传播算法解决如何高效利用 loss 的梯度优化各层参数的问题。

1.2.1 核心思想：为什么需要这四个公式？

分析

终极目标：通过更新参数 $\mathbf{W}$ 和 $\mathbf{b}$ 来降低损失 E 。

梯度下降的直觉：对某个参数 θ （可以是 $w_{ij}^{(l)}$ 或 $b_i^{(l)}$ ）：

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial E}{\partial \theta}$$

- 若 $\partial E / \partial \theta > 0$ ：增加 θ 会增大损失 → 需要减小 θ （往负方向调）
- 若 $\partial E / \partial \theta < 0$ ：增加 θ 会减小损失 → 需要增大 θ （往正方向调）

因此，核心任务就是**计算所有参数的梯度** $\partial E / \partial w_{ij}^{(l)}$ 和 $\partial E / \partial b_i^{(l)}$ 。

简化的关键：在推导过程中，我们发现所有梯度都可以统一表示为“某层误差项 $\delta_i^{(l)}$ ”的简单函数。因此，我们转而先计算每一层的 δ （BP-1 和 BP-2），再用

它们一步算出所有梯度（BP-3 和 BP-4）。

1.2.2 变量定义

在理论推导部分统一用单样本视角考虑问题，

首先统一符号。考虑第 l 层：

- $\mathbf{z}^{(l)}$ ：该层的**加权输入**向量， $z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$
- $\mathbf{a}^{(l)}$ ：该层的**激活输出**向量， $\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$
- $\mathbf{W}^{(l)}$ ：权重矩阵， $w_{ij}^{(l)}$ 表示第 $l-1$ 层第 j 个神经元到第 l 层第 i 个神经元的连接
- $\mathbf{b}^{(l)}$ ：偏置向量

定义

定义 1.2.1 (误差项 δ)。定义第 l 层第 i 个神经元的**误差项**为损失 E 对该神经元加权输入 $z_i^{(l)}$ 的偏导数：

$$\delta_i^{(l)} = \frac{\partial E}{\partial z_i^{(l)}}$$

备注

直觉理解： $\delta_i^{(l)}$ 衡量了该神经元对最终损失的“责任”大小——绝对值越大，意味着改变该神经元对减少损失的效果越显著。整个反向传播的核心任务，就是高效地计算出所有层的 δ 。

1.2.3 四个核心公式的推导

以下推导均针对**单个样本**，多样本（mini-batch）情形仅需对梯度取平均，公式形式不变。

1.2.3.1 BP-1：输出层误差

根据定义，输出层（第 L 层）的误差项为：

$$\delta_i^{(L)} = \frac{\partial E}{\partial z_i^{(L)}} = \frac{\partial E}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = \frac{\partial E}{\partial a_i^{(L)}} \cdot f'(z_i^{(L)})$$

向量化形式：

$$\delta^{(L)} = \left(\frac{\partial E}{\partial \mathbf{a}^{(L)}} \right) \odot f'(\mathbf{z}^{(L)})$$

分析

与损失函数的联系： $\partial E / \partial \mathbf{a}^{(L)}$ 取决于第 1 节定义的损失函数。

- 若使用 MSE 损失 $\mathcal{L} = \frac{1}{2} \|\mathbf{a}^{(L)} - \mathbf{y}\|_2^2$ ，则：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} = \mathbf{a}^{(L)} - \mathbf{y}$$

- 若使用交叉熵损失 $\mathcal{L} = -\sum_k y_k \log a_k$ 与 Softmax $a_k = e^{z_k} / \sum_j e^{z_j}$ 配合时，链式法则展开为：

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_k \left(-\frac{y_k}{a_k} \right) \cdot \frac{\partial a_k}{\partial z_i}$$

其中 Softmax 的导数为 $\partial a_k / \partial z_i = a_k(\delta_{ki} - a_i)$ 。考虑到 one-hot 编码特性， $\sum y_k = 1$ ，代入后，除 $a_i - y_i$ 外所有项均抵消，故得：

$$\delta^{(L)} = \mathbf{a} - \mathbf{y}$$

1.2.3.2 BP-2：隐藏层误差

由前向传播公式，

$$z_j^{(l+1)} = \sum_k w_{jk}^{(l+1)} a_k^{(l)} + b_j^{(l+1)}, \quad a_k^{(l)} = f(z_k^{(l)})$$

推导：由 $\delta_i^{(l)} = \partial E / \partial z_i^{(l)}$ ，用链式法则展开：

$$\delta_i^{(l)} = \sum_j \frac{\partial E}{\partial z_j^{(l+1)}} \cdot \frac{\partial z_j^{(l+1)}}{\partial z_i^{(l)}} = \sum_j \delta_j^{(l+1)} \cdot \frac{\partial z_j^{(l+1)}}{\partial a_i^{(l)}} \cdot \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} = \sum_j \delta_j^{(l+1)} \cdot w_{ji}^{(l+1)} \cdot f'(z_i^{(l)})$$

向量化形式：

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \right) \odot f'(\mathbf{z}^{(l)})$$

分析

误差信号 δ 通过权重矩阵的转置从后一层“传回”前一层，再乘上当前层激活函数的导数，完成一次反向传播。

1.2.3.3 BP-3: 权重梯度

根据前向传播的定义，第 l 层第 i 个神经元的加权输入为：

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

因此 $z_i^{(l)}$ 对权重 $w_{ij}^{(l)}$ 的偏导数为：

$$\frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} = a_j^{(l-1)}$$

利用链式法则，损失 E 对权重 $w_{ij}^{(l)}$ 的梯度为：

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \frac{\partial E}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} \cdot a_j^{(l-1)}$$

向量化形式：

$$\boxed{\frac{\partial E}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T}$$

备注

直觉：权重梯度由两个因子共同决定——当前层神经元的“责任” $\delta_i^{(l)}$ 和上一层神经元的“活跃度” $a_j^{(l-1)}$ 。若前一层神经元未被激活 ($a_j^{(l-1)} \approx 0$)，则该权重不会被调整。

1.2.3.4 BP-4: 偏置梯度

根据前向传播的定义：

$$z_i^{(l)} = \sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)}$$

因此 $z_i^{(l)}$ 对偏置 $b_i^{(l)}$ 的偏导数为：

$$\frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = 1$$

利用链式法则，损失 E 对偏置 $b_i^{(l)}$ 的梯度为：

$$\frac{\partial E}{\partial b_i^{(l)}} = \frac{\partial E}{\partial z_i^{(l)}} \cdot \frac{\partial z_i^{(l)}}{\partial b_i^{(l)}} = \delta_i^{(l)} \cdot 1 = \delta_i^{(l)}$$

向量化形式：

$$\frac{\partial E}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)}$$

备注

直觉：偏置的梯度就等于该神经元自身的误差项 $\delta_i^{(l)}$ ，无需额外乘法操作。这也解释了为何偏置在更新时通常使用与权重相同的学习率。